AI-Guided Prioritization of Protein Mutations for Stability and Functional Retention

Medvolt.AI — AI Research Scientist case study Dataset: `Protein_Mutation_Assignment_Dataset` (fully synthetic) · Random seed: `20261102` Environment: Python 3.11.15, scikit-learn 1.9.1 · 487 automated tests, `ruff` and `mypy` clean Extended version with full derivations and all candidate write-ups: `report/final_report_full.md`

---

1. Problem and objective

A 300-residue protein admits 5,700 single-point substitutions; a wet-lab campaign can characterise tens. The binding constraint is assay throughput, so the question is not "can we predict mutation effects" but "which handful should enter the queue next".

That question is irreducibly **multi-objective**. A mutation must be thermodynamically tolerated and must retain function, and the two trade off: the most stabilizing substitutions often sit at buried positions that disturb an active site, while the most functionally conservative changes are often thermodynamically uninteresting.

This system predicts `stability_ddG` (negative = stabilizing) and `functional_score` (1 = wild-type-like) for unlabeled mutations, estimates **per-candidate uncertainty**, flags out-of-distribution candidates, and returns a ranked, justified Top 10.

The emphasis throughout is **honest performance estimation**. For a system that decides which molecules get synthesized, an over-optimistic validation score is worse than a pessimistic one: it converts a modelling artefact into wasted bench time. Section 3 shows the obvious validation choice inflates performance by up to 0.14 Spearman; Section 5 shows the most expensive feature block contributes almost nothing.

Table	Rows	Proteins	Families
`labeled_mutations.csv`	840	28	7 (PF01–PF07)
`candidate_mutations_unlabeled.csv`	180	35 (8 unseen)	9 (PF08, PF09 unseen)
`protein_embeddings.csv`	36	36	9

Two structural facts drive every subsequent decision. **Mutations are nested within proteins, proteins within families** — 840 mutations cover only 28 proteins, so rows are not independent. And **the candidate set is partly out-of-distribution by construction**: 58 of 180 candidates (32%) belong to families with no labeled examples at all.

---

2. Data, preprocessing and quality control

Implemented in `src/data_processing.py` (58 tests).

The mutation tables join to the embedding table on `protein_id` (many-to-one). The join is verified, not assumed: cardinality is asserted via `validate="many_to_one"`; zero row loss is enforced (840 → 840, 180 → 180); vector alignment is re-checked element-wise after the merge, which catches a misaligned join that preserved row counts; and `protein_family` / `protein_length`, present in both tables, are confirmed to agree per protein before one copy is dropped.

The quality audit scans for nulls, infinities and violations of the documented ranges. **The dataset is clean: zero missing, zero infinite, zero out-of-range values.** The handling policy is nonetheless implemented and tested, because a clean dataset today is no guarantee for a refresh: ±inf → NaN then imputed; a missing *target* drops the row (imputing a label fabricates supervision); a missing *feature* takes the median fitted on labeled data only; out-of-range values are logged but **not clipped**, since silent correction hides data problems. Every action is recorded in `data/processed/feature_manifest.json`.

Feature engineering. Secondary structure (H/E/C) is one-hot encoded against a fixed category list, so training and candidate matrices always share identical columns in identical order. Physicochemical deltas (Δhydrophobicity, Δcharge, Δvolume) are produced in both signed and absolute form: the signed delta carries *direction* — a hydrophobic-to-polar change behaves very differently in the core than on the surface — while the absolute delta carries *magnitude of*

perturbation, which a tree would otherwise need two splits to express. Relative position (`position / protein_length`) makes sequence location comparable across proteins. Anomaly handling runs **before** delta computation, so an imputed component yields a consistent delta. Output: 22 tabular + 256 embedding features.

---

3. Validation strategy and leakage prevention

Implemented in `src/validation.py` (46 tests).

The leakage mechanism. Standard K-fold partitions the 840 mutations at random, so for almost every fold there are training and validation rows from the *same protein*. The model can then score well by recognising the protein rather than learning the effect of the substitution — and the protein embedding, being constant within a protein, is a perfect protein identifier.

Three nested regimes were implemented and compared: `random` (no grouping), `protein_group` (transfer to an unseen protein in a seen family) and `family_group` (transfer to an unseen family — the candidates' regime). Every generated fold set is verified before use: each row validated exactly once, no row on both sides, no shared group in grouped strategies. A violation raises `LeakageError` rather than producing a quietly optimistic number.

Empirical leakage, measured on the labeled set:

Strategy	Val rows/fold	Proteins on both sides	Families on both sides
`random`	168 (exactly)	**28 of 28**	**7 of 7**
`protein_group`	155–183	0	up to 5
`family_group`	106–246	0	0

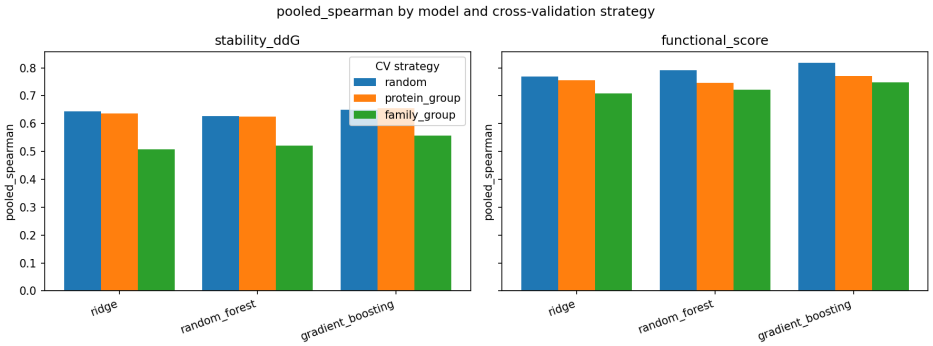
Random K-fold places *every* protein and *every* family on both sides of *every* fold. The leakage is not marginal; it is total.

The optimism gap — pooled out-of-fold Spearman for the selected gradient-boosted model:

Target	`random`	`protein_group`	`family_group`	Gap
`stability_ddG`	0.650	0.656	**0.556**	**+0.094**
`functional_score`	0.817	0.771	**0.747**	**+0.070**

Across all three models the gap reaches **+0.137** (ridge, stability). Generalising to a new protein within a *known* family is nearly free (`protein_group` ≈ `random`); generalising to a new *family* is where performance is actually lost — precisely the regime of 32% of the candidates. `family_group` is therefore the **primary** strategy for model selection, the ablation and all headline figures.

One caveat: with only 7 labeled families, 5-fold grouping yields 1–2 held-out families per fold and uneven fold sizes (106–246 rows). All headline metrics are therefore **pooled across all out-of-fold predictions** rather than averaged over folds, which weights every mutation equally.



Pooled Spearman by model and cross-validation strategy. Bars fall from left to right within every model as the split grows stricter: that fall is the optimism a random split would have bought.

---

4. Modelling approach and architecture

Architecture in `src/model.py` (16 tests); fold-wise execution in `src/train.py` (14 tests); metrics, selection and the optimism gap in `src/evaluate.py` (22 tests).

Each target is modelled by independent single-target regression. Multi-output regression was rejected deliberately: the two targets have different scales, different error structure and different achievable accuracy, and nothing requires a shared representation.

Every fold fits one scikit-learn `Pipeline` from scratch:

```
ColumnTransformer
  ■■ tabular   (22 cols) : SimpleImputer(median) → StandardScaler
  ■■ embedding (256 cols): SimpleImputer(median) → StandardScaler → PCA(16)
→ GradientBoostingRegressor
```

Because imputation, scaling and PCA are **steps inside the pipeline**, `fit` on the training fold is the only place their statistics are learned; the validation fold reaches `predict` only. This is asserted rather than assumed: fitted imputer, scaler and PCA statistics are confirmed equal to the training fold's and *different* from whole-table statistics, so the test has teeth; and multiplying validation features by 1000 while setting validation targets to –999 leaves every fitted transformer and model coefficient bit-identical.

Model selection used pooled out-of-fold Spearman under `family_group`, chosen because the deliverable is a *ranking*: Spearman measures the ordering quality Top-10 selection depends on, while RMSE would reward calibration the downstream task never uses. Gradient boosting won both targets (`stability_ddG`: 0.507 ridge / 0.522 forest / **0.556** boosting; `functional_score`: 0.708 / 0.721 / **0.747**), consistently across all three strategies, so the choice is not an artefact of one split.

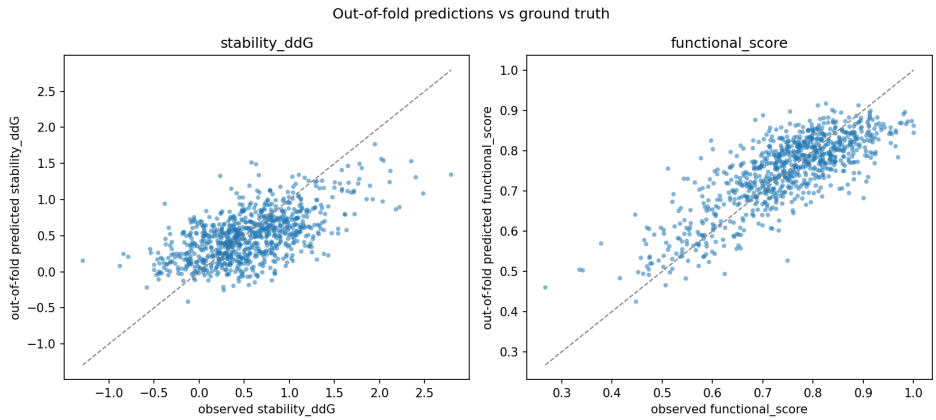
---

5. Results and ablation findings

Selected model, pooled out-of-fold under `family_group`:

Target	RMSE	R^2	Spearman	Fold SD
`stability_ddG`	0.426	0.341	0.556	0.052
`functional_score`	0.069	0.634	0.747	0.031

Functional retention is substantially more predictable than stability (R^2 0.63 vs 0.34). With RMSE 0.426 on a target whose predicted range is –0.149 to +1.41, the model resolves *coarse* stability ordering but cannot distinguish candidates differing by less than roughly 0.4 kcal/mol.



Out-of-fold predictions against ground truth for the selected models under family-grouped CV. The compression of predicted `stability_ddG` relative to its observed range is the visual form of the R^2 = 0.34 above.

Ablation (`src/ablation.py`, 21 tests). Each subset is cross-validated from scratch under `family_group` with folds, seed and model held fixed, so differences are attributable to features alone. Pooled out-of-fold Spearman, with change against the full model:

Subset	Features	stability_ddG	Δ	functional_score	Δ
D_full	22 + 256	**0.556**	—	**0.747**	—
E_all_tabular_no_embeddings	22 + 0	0.546	–0.010	0.723	–0.025
F_full_minus_deltas	16 + 256	0.536	–0.021	0.717	–0.030
G_full_minus_structural	15 + 256	0.413	–0.143	0.370	–0.377
C_tabular_plus_embeddings	9 + 256	0.397	–0.159	0.327	–0.421
A_tabular_no_embeddings	9 + 0	0.373	–0.183	0.271	–0.477
B_embeddings_only	0 + 256	–**0.205**	–0.762	–**0.110**	–0.857

(1) The structural microenvironment carries almost all the signal. Removing `structural` costs –0.143 and –0.377 — by far the largest effect. Conservation, solvent accessibility and distance to active site describe the local constraint the substitution must satisfy, which is biophysically what one would expect.

(2) The 256-dimensional embeddings contribute essentially nothing, and alone are worse than useless. Adding them to the full tabular set buys +0.010 and +0.025; used alone they give **negative** rank correlation (–0.205, –0.110). The reason is structural, not a bug: the embedding is a **per-protein constant**, identical for every mutation within a protein. It cannot discriminate between mutations in the same protein — the actual task — and can only shift a family-level baseline, which under family-grouped CV is unseen and generalises poorly. The implication is direct: **subset E (22 features, no embeddings, no PCA) performs within 0.01–0.03 Spearman of the full 278-feature model** at a fraction of the cost. Retaining the embeddings is defensible only as headroom for a mutation-aware representation (Section 8).

(3) Engineered delta features earn their place, modestly — removing them costs –0.021 and –0.030, small but consistent, and larger than the embeddings' contribution.

---

6. Uncertainty and prioritization

Implemented in `src/predict.py` (35 tests).

The five `family_group` fold pipelines per target form an ensemble; the mean is the point estimate and the sample standard deviation ($K-1$) is the uncertainty. Each fold model carries its own fitted imputer, scaler and PCA, so no transformer is reused across folds.

Objectives are **rank-normalised within the candidate pool**, oriented so higher is better, then combined:

$$\text{priority}(c) = w_{\text{stab}}\,\widetilde{(-\Delta\Delta G)}(c) + w_{\text{func}}\,\widetilde{S_{\text{func}}}(c) - \lambda\,\widetilde{u}(c), \qquad u(c) = \sqrt{s_{\text{stab}}^2 + s_{\text{func}}^2}$$

with $w_{\text{stab}} = w_{\text{func}} = 0.5$ and $\lambda = 0.25$. The stability term uses $-\Delta\Delta G$ because negative ddG is stabilizing. Uncertainties combine in quadrature, and the penalty applies to the *normalised* uncertainty so it is commensurate with the objective terms.

A negative result that matters. Ensemble spread does **not** detect distribution shift:

Flag	n	Mean s_{stab}	Mean s_{func}
`in_distribution`	122	0.1031	0.0160
`unseen_family`	58	**0.0969**	**0.0146**

Fold disagreement is *slightly lower* for out-of-distribution candidates: the five fold models agree with one another on novel families while potentially all being wrong together — exactly the shared error a disagreement-based estimate cannot see. **The uncertainty penalty therefore does not protect against domain shift; only the explicit `ood_flag` does.**

Final Top 10

Rank	Mutation	Protein	Family	Sub.	SS	ddG	±s	Func	±s	Score	Flag
1	CAND0002	PROT007	PF02	M496W	E	+0.234	0.040	0.878	0.013	0.885	in_dist
2	CAND0034	PROT001	PF01	W11Y	C	+0.252	0.020	0.865	0.014	0.865	in_dist
3	CAND0148	PROT022	PF06	S490L	H	−0.013	0.085	0.861	0.027	0.850	in_dist
4	CAND0121	PROT006	PF02	I66V	H	+0.172	0.078	0.885	0.006	0.846	in_dist
5	CAND0159	PROT004	PF01	C14V	E	+0.111	0.070	0.851	0.006	0.842	in_dist
6	CAND0175	PROT018	PF05	C475M	C	+0.263	0.060	0.905	0.007	0.835	in_dist
7	CAND0162	PROT012	PF03	Y12F	C	−**0.149**	0.038	0.800	0.013	0.833	in_dist
8	CAND0047	PROT019	PF05	V196M	C	+0.295	0.062	0.873	0.007	0.806	in_dist
9	CAND0120	PROT004	PF01	I130M	H	+0.251	0.072	0.869	0.007	0.806	in_dist
10	CAND0119	PROT007	PF02	T191F	C	+0.088	0.037	0.798	0.011	0.801	in_dist

No out-of-distribution candidate reached the Top 10, despite 32% of the pool being unseen-family. That is favourable for reliability, but it is an emergent property of the scoring, not an enforced constraint: if exploring novel families is a campaign goal, those candidates must be requested explicitly from `candidate_predictions.csv`.

The shortlist is driven mainly by functional retention. Eight of ten are predicted destabilizing; across the whole pool only 5 of 180 candidates are predicted stabilizing at all. For most of the list the stability objective is in practice selecting the **least destabilizing** option, and predicted retention is tightly bunched (0.798–0.905).

---

7. Scientific interpretation

Generated by `src/interpret.py` (24 tests); all candidates in `results/mutation_rankings/candidate_interpretation.md`. Bands are the pool's 33%/67% quantiles. The notes state general biophysical expectations to make the inputs interpretable; they are **not** feature attributions, as none were computed.

Rank 1 — CAND0002 (PF02): M496W, β-strand. ddG **+0.234 ± 0.040**, function **0.878 ± 0.013**. Conservation 0.232 (lower third), solvent accessibility 0.622 (upper third), 27.5 Å from the active site, Δvolume **+64.9 Å³**. The profile is coherent: low conservation, high exposure and distance from the catalytic centre jointly predict high retention, and the ensemble is unusually tight (±0.013). The bulky indole of Met→Trp is the likely source of the mild destabilization — at an exposed site a +65 Å³ increase is far better tolerated than in a packed core. **A functional-retention pick, not a stability pick.**

Rank 3 — CAND0148 (PF06): S490L, helix. ddG −**0.013 ± 0.085**, function **0.861 ± 0.027**. This needs the most careful reading in the list: the ddG magnitude is roughly **six times smaller than its own ensemble spread**, so **the sign is not resolved** — the honest statement is "approximately neutral", not "stabilizing". It is a polar→hydrophobic substitution (Δhydrophobicity +4.6) with a large volume increase (+77.7 Å³) on an *exposed* helix face, where burying a large hydrophobic residue is entropically unfavourable; mild destabilization would be the more expected outcome. It carries the highest uncertainty on both targets of any top candidate and is **the entry most in need of measurement rather than trust**.

Rank 7 — CAND0162 (PF03): Y12F, coil. ddG −**0.149 ± 0.038**, function **0.800 ± 0.013**. **The only Top-10 candidate whose stabilizing prediction is resolved** — the magnitude is about four times the ensemble spread. Y→F removes a single hydroxyl while preserving the aromatic ring, so it is near-isosteric (Δvolume −3.7 Å³); at a nearly fully exposed position (SASA 0.940) losing the polar hydroxyl raises hydrophobicity without disturbing packing. The cost is function: 0.800 is the second-lowest retention in the list, which pushes it to rank 7 under equal weighting. **Were the campaign weighted toward stability, this would be the lead candidate** — changing `prioritization.weights` in config

reorders the list without touching any code.

Cross-cutting. The top candidates share low-to-moderate conservation (0.178–0.414, against a pool median near 0.44) and moderate-to-high exposure (0.447–0.940). The model has learned a recognisable rule — *perturb loosely constrained, exposed positions* — which is the heuristic a protein engineer would apply manually. Two cautions: **Δcharge is 0.0 for every top pick**, and whether that is a learned preference or a property of the pool is not determinable from these cases; and **predicted retention spans only 0.798–0.905 across the Top 10**, narrower than that target's own RMSE (0.069), so single-rank ordering is not meaningful. The defensible claim is that these ten are collectively better than the rest of the pool, not that rank 1 beats rank 4.

---

8. Limitations and future directions

Limitations. Every number describes a **synthetic** dataset; the relationships are generator artefacts and no biophysical conclusion transfers. What transfers is the methodology. **7 labeled families is too few for stable family-grouped CV** — a ±0.05 Spearman difference between models should not be over-interpreted. **Stability is only coarsely predicted** (R^2 0.34); differences below ~0.4 kcal/mol are unresolved. **Uncertainty is underestimated**: ensemble spread quantifies model disagreement, not total predictive error, and is blind to domain shift. **Scores are pool-relative**, not an absolute measure of merit. **No feature attribution** was computed, so Section 7 reasons from input values and general biophysics rather than per-prediction attributions — SHAP on the selected model is the single cheapest improvement available. Finally, results are **environment-sensitive**: bit-reproducible for a fixed environment, but a scikit-learn change reorders the Top 10. Pinned versions and seed are in `results/models/training_metadata.json`.

Future directions, in priority order. *First, replace the per-protein embedding with a mutation-aware representation.* The embedding's failure is entirely explained by its being constant within a protein; per-*residue* representations from an ESM-style protein language model give a position-specific vector, and the standard mutation-effect signals — wild-type-vs-mutant log-odds at the mutated position, and pseudo-log-likelihood difference — are strong zero-shot predictors precisely because they vary per mutation. This addresses the bottleneck the ablation identified rather than adding capacity elsewhere.

Second, structure-aware modelling. Given that the structural block dominates the ablation, better structural features are the most promising axis: with AlphaFold or ESMFold structures, a graph neural network over the residue contact graph with the mutated residue as query node would capture packing and local environment that `solvent_accessibility` and `distance_to_active_site` only approximate.

Third, honest uncertainty. Replace fold disagreement with something sensitive to domain shift — deep ensembles from different initialisations, evidential regression, or conformal prediction for calibrated intervals with coverage guarantees — plus a distance-to-training-data measure giving a continuous OOD score rather than a binary flag.

Fourth, validation at scale and active learning. With more families, leave-one-family-out CV would give a far more stable estimate, and nested CV would remove the optimistic bias of tuning on the selection metric. Closing the loop with active learning requires the improved uncertainty first: an acquisition function built on a measure blind to novelty would systematically under-explore exactly the novel families the campaign most needs to learn about.

Appendix: reproducing these results

```
python3.11 -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
python -m pytest -q                          # 487 tests
python main.py --config config/config.yaml   # whole pipeline (~15 min, BLAS-dependent)
python report/build_pdf.py                   # regenerate this PDF from its Markdown source
```

Individual stages run as `python -m src.<stage>` for `data_processing`, `validation`, `train`, `evaluate`, `predict`, `ablation` and `interpret`; `python main.py --from <stage>` resumes from any point. Requires Python ≥ 3.11 (see `requirements.txt` for why 3.10 is unsupported). All configuration lives in `config/config.yaml`; dataset facts and naming conventions in `src/common/constants.py`; engineering standards in `docs/CODING_GUIDELINES.md`.